

COMPTE RENDU

SDN & Network Softwarization — TP1 — Construire un mini-réseau
Linux sans Mininet
4e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2025/2026

Enseignante : M. BOUHORMA

Date : 30 septembre 2026

Table des matières

Cadre du TP	2
1 Ce que vous allez apprendre	3
2 Préparation de l'environnement	4
3 État initial : observer le réseau de la machine	5
4 Étape A — créer deux hôtes isolés et un câble virtuel	6
4.1 Créer les deux network namespaces	6
4.2 Créer la paire veth	7
4.3 Déplacer une extrémité dans chaque namespace	7
4.4 Adresser et activer les interfaces	8
4.5 Tester la communication	8
5 Observer ARP puis ICMP	10
6 Étape B — remplacer le câble direct par un switch Linux	11
6.1 Créer les namespaces et le bridge	11
6.2 Créer deux « câbles » vers le switch	12
6.3 Adresser les hôtes	13
6.4 Vérifier le « câblage »	13
6.5 Tester et observer la table MAC	14
7 Analyse de trafic sur le bridge	15
8 Faire circuler un vrai service HTTP	16
9 Mini-défis	17
9.1 Défi 1 — ajouter un troisième namespace h3 connecté à br0	17
9.2 Défi 2 — vérifier que h3 joint h1 et h2	18
9.3 Défi 3 — vider le cache neighbor de h3 et observer le ping suivant	19
9.4 Défi 4 — h3 avec l'adresse 10.21.0.3/24, sans routeur	20
10 Nettoyage obligatoire	21
11 Synthèse : le lien direct avec Mininet	21
12 Grille de validation rapide	22

Cadre du TP

Niveau	Durée conseillée	Plateforme
Master — préparation SDN	1 h 45	Ubuntu 24.04 LTS (accès sudo)
Mode	Approche	Livrable
Individuel	Construction manuelle, sans Mininet	Compte rendu avec captures

TABLE 1 – Cadre général du TP

Question fil rouge : *comment faire communiquer deux « machines » virtuelles dans un seul Ubuntu, sans créer deux nouvelles VM ?*

Objectif final : construire à la main la topologie

h1 ↔ veth ↔ br0 ↔ veth ↔ h2

entièrement à l'intérieur de la machine Ubuntu, avec les mécanismes Linux que Mininet automatisera ensuite.

Règle de travail :

- Ne pas se limiter à « ping fonctionne / ne fonctionne pas » : observer et interpréter (ARP, ICMP, table MAC) ;
- N'utiliser que des interfaces virtuelles créées pour le laboratoire ;
- Ne jamais ajouter l'interface réelle de la machine au bridge du TP.

Listing 1 – Commandes à maîtriser durant le TP

```
1 ip netns    ip link    ip addr    ip -n <ns>    ip netns exec
2 bridge link  bridge fdb show    ip neigh    ping    tcpdump    curl
3 python3 -m http.server
```

Note sur l'environnement. Le sujet est rédigé pour une VM Ubuntu (interfaces `ens33`, `eth0`, `enp0s3`...). Les manipulations de ce compte rendu ont été faites directement sur mon portable Ubuntu 24.04 (*Taha-inspiron-16*), connecté au Wi-Fi du campus. L'interface réelle est donc `wlp0s20f3` (10.85.100.55/16). Comme tout le TP repose sur des interfaces virtuelles (namespaces, veth, bridge), cette différence ne change pas les commandes, et l'interface `wlp0s20f3` n'a jamais été modifiée.

1 Ce que vous allez apprendre

Le TP fait construire à la main trois mécanismes fondamentaux de Linux, utilisés dans l'émulation réseau.

Mécanisme	Rôle
Network namespace	Espace réseau isolé, avec ses propres interfaces, adresses IP, routes et table de voisinage.
Paire veth	Câble Ethernet virtuel à deux extrémités, chacune étant une interface Linux.
Bridge Linux	Commutateur Ethernet logiciel de niveau 2 auquel on raccorde plusieurs hôtes virtuels.

TABLE 2 – Les trois mécanismes étudiés

Topologie finale obtenue à l'issue des mini-défis (h3 y est ajouté en plus de h1 et h2) :

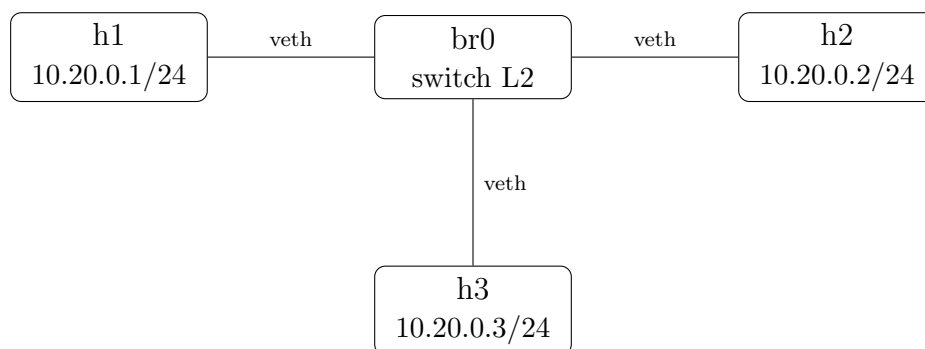


FIGURE 1 – Topologie finale : trois hôtes reliés au bridge `br0`

2 Préparation de l'environnement

Listing 2 – Vérification des outils

```
1 sudo apt update
2 sudo apt install -y iproute2 iputils-ping tcpdump curl python3
3 ip -V
4 ping -V | head -n 1
5 python3 --version
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip -V
ip utility, iproute2-6.1.0, libbpf 1.3.0
```

FIGURE 2 – Sortie de `ip -V`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ping -V | head -n 1
ping from iputils 20240117
```

FIGURE 3 – Sortie de `ping -V | head -n 1`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ python3 --version
Python 3.12.3
```

FIGURE 4 – Sortie de `python3 --version`

Outil	Version relevée
ip	iproute2-6.1.0, libbpf 1.3.0
ping	iputils 20240117
python3	Python 3.12.3

TABLE 3 – Versions des outils disponibles

Les commandes `ip`, `ping` et `python3` sont disponibles ; `tcpdump` et `curl` sont utilisés plus loin dans le TP. Ce TP ne modifie pas l'interface réseau principale de la machine.

3 État initial : observer le réseau de la machine

Listing 3 – Observation de l'état initial

```
1 ip -br addr
2 ip route
3 ip link
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip -br addr
lo                UNKNOWN          127.0.0.1/8 :::1/128
wlp0s20f3         UP                10.85.100.55/16 fe80::f7dd:c516:6783:a940/64
virbr0            DOWN             192.168.122.1/24
docker0          DOWN             172.17.0.1/16
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 5 – Sortie de `ip -br addr`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip route
default via 10.85.0.5 dev wlp0s20f3 proto dhcp src 10.85.100.55 metric 600
10.85.0.0/16 dev wlp0s20f3 proto kernel scope link src 10.85.100.55 metric 600
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.122.0/24 dev virbr0 proto kernel scope link src 192.168.122.1 linkdown
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 6 – Sortie de `ip route`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DORMANT group default qlen 1000
   link/ether 60:a5:e2:64:75:d0 brd ff:ff:ff:ff:ff:ff
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default qlen 1000
   link/ether 52:54:00:9e:3b:49 brd ff:ff:ff:ff:ff:ff
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default
   link/ether 46:2b:87:5b:cf:ab brd ff:ff:ff:ff:ff:ff
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 7 – Sortie de `ip link`

Interface	État	Adresse / rôle
lo	UNKNOWN	127.0.0.1/8 — boucle locale
wlp0s20f3	UP	10.85.100.55/16 — carte Wi-Fi, seule interface reliée à l'extérieur
virbr0	DOWN (NO-CARRIER)	192.168.122.1/24 — pont libvirt/KVM
docker0	DOWN (NO-CARRIER)	172.17.0.1/16 — pont Docker

TABLE 4 – Interfaces présentes avant le TP

Questions de compréhension

1. Quelle interface possède l'adresse IP permettant à la machine de communiquer avec le réseau extérieur ?

C'est `wlp0s20f3` (carte Wi-Fi), qui porte `10.85.100.55/16`. C'est la seule interface à l'état UP avec une porteuse (`LOWER_UP`) : `virbr0` et `docker0` sont en `NO-CARRIER`, aucun conteneur ni VM ne les utilise pour l'instant.

2. Quelle est la route par défaut ?

`default via 10.85.0.5 dev wlp0s20f3 proto dhcp src 10.85.100.55 metric 600`. Tout paquet qui ne correspond à aucun réseau directement connecté est envoyé à la passerelle `10.85.0.5` par l'interface Wi-Fi. Le `proto dhcp` indique que cette route a été apprise automatiquement via le DHCP du réseau.

3. Pourquoi ne devons-nous pas modifier cette interface pendant ce TP ?

Cette interface porte la connectivité de la machine : la modifier (changer son adresse, la mettre down, l'ajouter à un bridge) couperait l'accès à Internet, au DNS et à toute session distante. Le TP se fait donc uniquement avec des interfaces virtuelles créées pour l'occasion, qui n'ont aucun impact sur le réseau réel. Dans mon cas, c'est encore plus important : une interface Wi-Fi en mode station ne peut en général pas être attachée à un bridge Linux.

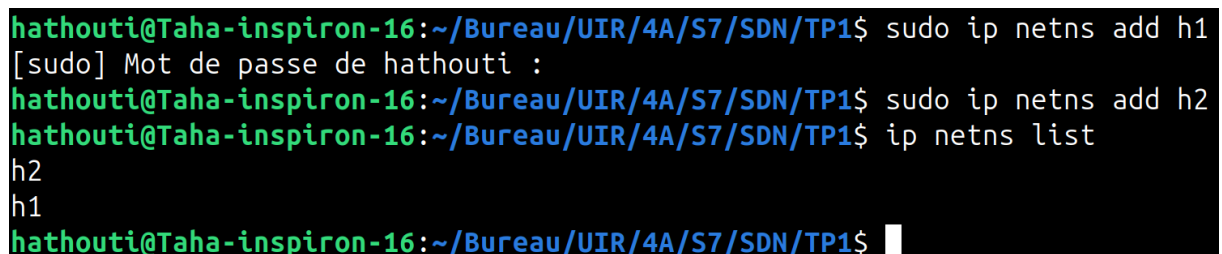
4 Étape A — créer deux hôtes isolés et un câble virtuel

Idée : une paire `veth` se comporte comme les deux extrémités d'un câble Ethernet entre deux hôtes isolés (`h1` et `h2`), sans switch.

4.1 Créer les deux network namespaces

Listing 4 – Création des namespaces

```
1 sudo ip netns add h1
2 sudo ip netns add h2
3 ip netns list
```



```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns add h1
[sudo] Mot de passe de hathouti :
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns add h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip netns list
h2
h1
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 8 – Création des namespaces `h1` et `h2` et vérification par `ip netns list`

`h1` et `h2` apparaissent dans la liste. Chacun possède sa propre pile réseau, mais aucune interface utile n'y est encore connectée.

4.2 Créer la paire veth

Listing 5 – Création de la paire veth

```
1 sudo ip link add veth-h1 type veth peer name veth-h2
2 ip -br link | grep veth
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link add veth-h1 type veth peer name veth-h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip -br link | grep veth
veth-h2@veth-h1 DOWN          d2:6a:ea:c2:50:9c <BROADCAST,MULTICAST,M-DOWN>
veth-h1@veth-h2 DOWN          5a:89:b7:6c:6a:29 <BROADCAST,MULTICAST,M-DOWN>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 9 – Création de la paire `veth-h1` / `veth-h2` dans le namespace de la machine

Les deux extrémités existent, sont liées (`veth-h2@veth-h1` et `veth-h1@veth-h2`) et sont encore DOWN. Elles ont chacune leur adresse MAC (`5a:89:b7:6c:6a:29` pour `veth-h1`, `d2:6a:ea:c2:50:9c` pour `veth-h2`). Une paire veth fonctionne comme un câble : tout paquet entrant d'un côté ressort de l'autre.

4.3 Déplacer une extrémité dans chaque namespace

Listing 6 – Déplacement des extrémités

```
1 sudo ip link set veth-h1 netns h1
2 sudo ip link set veth-h2 netns h2
3 sudo ip -n h1 link
4 sudo ip -n h2 link
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set veth-h1 netns h1
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set veth-h2 netns h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 link
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
7: veth-h1@if6: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether 5a:89:b7:6c:6a:29 brd ff:ff:ff:ff:ff:ff link-netns h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 link
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
6: veth-h2@if7: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
   link/ether d2:6a:ea:c2:50:9c brd ff:ff:ff:ff:ff:ff link-netns h1
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 10 – Déplacement des extrémités et vérification avec `ip -n h1 link` et `ip -n h2 link`

Chaque namespace ne voit plus que sa propre extrémité (`veth-h1` dans `h1`, `veth-h2` dans `h2`) ainsi que son `lo`, tous deux DOWN. Le champ `link-netns` indique dans quel namespace se trouve l'autre extrémité du câble.

4.4 Adresser et activer les interfaces

Listing 7 – Adressage et activation

```
1 sudo ip -n h1 addr add 10.10.0.1/24 dev veth-h1
2 sudo ip -n h2 addr add 10.10.0.2/24 dev veth-h2
3 sudo ip -n h1 link set lo up
4 sudo ip -n h2 link set lo up
5 sudo ip -n h1 link set veth-h1 up
6 sudo ip -n h2 link set veth-h2 up
7 sudo ip -n h1 -br addr
8 sudo ip -n h2 -br addr
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 addr add 10.10.0.1/24 dev veth-h1
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 addr add 10.10.0.2/24 dev veth-h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ █
```

FIGURE 11 – Attribution des adresses 10.10.0.1/24 et 10.10.0.2/24

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 link set lo up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 link set lo up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 link set veth-h1 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 link set veth-h2 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ █
```

FIGURE 12 – Activation de lo et des interfaces veth-h1 / veth-h2

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
veth-h1@if6       UP                10.10.0.1/24 fe80::5889:b7ff:fe6c:6a29/64
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
veth-h2@if7       UP                10.10.0.2/24 fe80::d06a:eaff:fec2:509c/64
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ █
```

FIGURE 13 – Vérification avec ip -n h1 -br addr et ip -n h2 -br addr

h1 possède 10.10.0.1/24 et h2 possède 10.10.0.2/24, les interfaces sont UP.

4.5 Tester la communication

Listing 8 – Tests de communication

```
1 sudo ip netns exec h1 ping -c 3 10.10.0.2
2 sudo ip netns exec h2 ping -c 3 10.10.0.1
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h1 ping -c 3 10.10.0.2
PING 10.10.0.2 (10.10.0.2) 56(84) bytes of data.
64 bytes from 10.10.0.2: icmp_seq=1 ttl=64 time=0.099 ms
64 bytes from 10.10.0.2: icmp_seq=2 ttl=64 time=0.052 ms
64 bytes from 10.10.0.2: icmp_seq=3 ttl=64 time=0.056 ms

--- 10.10.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2068ms
rtt min/avg/max/mdev = 0.052/0.069/0.099/0.021 ms
```

FIGURE 14 – Ping de h1 vers h2 (10.10.0.2)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h2 ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=0.092 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=0.058 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=64 time=0.089 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2041ms
rtt min/avg/max/mdev = 0.058/0.079/0.092/0.015 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 15 – Ping de h2 vers h1 (10.10.0.1)

Test	Paquets reçus	RTT moyen	Remarque
h1 → h2	3/3 (0 % de perte)	0,069 ms	ttl=64
h2 → h1	3/3 (0 % de perte)	0,079 ms	ttl=64

TABLE 5 – Résultats des pings de l'étape A

Les deux pings réussissent sans route par défaut : les deux adresses appartiennent au même sous-réseau 10.10.0.0/24. Le `ttl=64` inchangé confirme qu'aucun routeur n'a été traversé.

Questions de compréhension

1. Pourquoi un routeur n'est-il pas nécessaire ici ?

Les deux hôtes sont dans le même sous-réseau IP (10.10.0.0/24) et reliés directement au niveau Ethernet. Chacun voit l'autre comme « directement connecté » : il résout son adresse MAC par ARP et lui envoie la trame sans intermédiaire. Un routeur n'est nécessaire que pour passer d'un réseau IP à un autre, ce qui n'arrive pas ici.

2. Que représente physiquement la paire veth si l'on compare ce montage à un réseau réel ?

Un câble Ethernet reliant directement les cartes réseau de deux machines. Chaque extrémité de la paire correspond à une carte réseau (avec sa MAC) branchée à un bout du câble.

3. Un namespace est-il une VM complète ? Quel élément est réellement isolé dans ce TP ?

Non. Un namespace n'est pas une machine virtuelle : il n'a ni noyau ni système d'exploitation propres, il partage le noyau de la machine hôte. Seule la **pile réseau** est isolée : interfaces, adresses IP, table de routage, table de voisinage (ARP), règles de pare-feu et sockets/ports. Le système de fichiers et les utilisateurs restent partagés, ce que la section 8 illustre : le serveur lancé dans h1 sert les fichiers du dossier courant de la machine.

5 Observer ARP puis ICMP

Dans un premier terminal, la capture est lancée dans h1 ; dans le second, h2 envoie des pings vers h1.

Listing 9 – Capture ARP/ICMP sur veth-h1

```
1 sudo ip netns exec h1 tcpdump -ni veth-h1 'arp or icmp'
2 sudo ip netns exec h2 ping -c 2 10.10.0.1
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h1 tcpdump -ni veth-h1 'arp or icmp'
[sudo] Mot de passe de hathouti :
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on veth-h1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C15:23:39.005249 IP 10.10.0.2 > 10.10.0.1: ICMP echo request, id 55547, seq 1, length 64
15:23:39.005271 IP 10.10.0.1 > 10.10.0.2: ICMP echo reply, id 55547, seq 1, length 64
15:23:40.032315 IP 10.10.0.2 > 10.10.0.1: ICMP echo request, id 55547, seq 2, length 64
15:23:40.032359 IP 10.10.0.1 > 10.10.0.2: ICMP echo reply, id 55547, seq 2, length 64
15:23:44.448127 ARP, Request who-has 10.10.0.2 tell 10.10.0.1, length 28
15:23:44.448228 ARP, Request who-has 10.10.0.1 tell 10.10.0.2, length 28
15:23:44.448251 ARP, Reply 10.10.0.1 is-at 5a:89:b7:6c:6a:29, length 28
15:23:44.448258 ARP, Reply 10.10.0.2 is-at d2:6a:ea:c2:50:9c, length 28
15:24:00.676553 IP 10.10.0.2 > 10.10.0.1: ICMP echo request, id 55551, seq 1, length 64
15:24:00.676585 IP 10.10.0.1 > 10.10.0.2: ICMP echo reply, id 55551, seq 1, length 64
15:24:01.729203 IP 10.10.0.2 > 10.10.0.1: ICMP echo request, id 55551, seq 2, length 64
15:24:01.729231 IP 10.10.0.1 > 10.10.0.2: ICMP echo reply, id 55551, seq 2, length 64

12 packets captured
12 packets received by filter
0 packets dropped by kernel
```

FIGURE 16 – Capture tcpdump sur veth-h1 (terminal 1)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h2 ping -c 2 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=0.069 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=0.080 ms

--- 10.10.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1027ms
rtt min/avg/max/mdev = 0.069/0.074/0.080/0.005 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h2 ping -c 2 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=0.099 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=0.088 ms

--- 10.10.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1053ms
rtt min/avg/max/mdev = 0.088/0.093/0.099/0.005 ms
```

FIGURE 17 – Pings de h2 vers h1 (terminal 2)

Observation. La capture contient 12 paquets, dans cet ordre :

- 15:23:39 et 15:23:40 : un premier ping (Echo Request / Echo Reply, seq 1 et seq 2), **sans ARP avant** ;
- 15:23:44 : quatre paquets ARP, un **who-has** dans chaque sens (10.10.0.2 tell 10.10.0.1 et 10.10.0.1 tell 10.10.0.2) et les deux **Reply** correspondants ;
- 15:24:00 : un second ping, de nouveau uniquement en ICMP.

Les adresses MAC des réponses ARP (10.10.0.1 is-at 5a:89:b7:6c:6a:29, 10.10.0.2 is-at d2:6a:ea:c2:50:9c) sont exactement celles relevées avec `ip link` en étape A.

Interprétation. Avant d’envoyer une trame Ethernet à 10.10.0.1, h2 doit connaître l’adresse MAC correspondante : ARP construit ce lien IP ↔ MAC. Ici, l’ARP n’apparaît pas avant le premier ICMP parce que les pings de l’étape 4.5 avaient déjà rempli le cache de voisinage des deux hôtes. Les entrées, plus confirmées depuis un moment (état **STALE**), restent utilisables : l’ICMP part immédiatement avec la MAC en cache. L’utilisation de l’entrée la fait passer en **DELAY**, puis en **PROBE** environ 5 secondes plus tard, et le noyau la revalide par un ARP. Les deux hôtes le font chacun de leur côté, car h1 a utilisé son entrée pour répondre, d’où les deux **who-has**. Pour observer l’enchaînement classique « ARP puis ICMP », il faut partir d’un cache vide, ce que fait le mini-défi 3.

6 Étape B — remplacer le câble direct par un switch Linux

Le montage direct est supprimé pour créer une topologie plus proche de Mininet : deux hôtes reliés par un switch logiciel Linux (**br0**).

Listing 10 – Suppression du montage de l’étape A

```
1 sudo ip netns del h1
2 sudo ip netns del h2
3 sudo ip link del br0 2>/dev/null || true
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns del h1
[sudo] Mot de passe de hathouti :
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns del h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link del br0 2>/dev/null || true
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 18 – Suppression de l’étape A

6.1 Créer les namespaces et le bridge

Listing 11 – Namespaces et bridge

```
1 sudo ip netns add h1
2 sudo ip netns add h2
3 sudo ip link add br0 type bridge
4 sudo ip link set br0 up
5 ip -br link show br0
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns add h1
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns add h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 19 – Création des namespaces h1 et h2

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link add br0 type bridge
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set br0 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 20 – Création du bridge br0 et activation

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip -br link show br0
br0                DOWN                9a:ee:05:93:7b:54 <NO-CARRIER,BROADCAST,MULTICAST,UP>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 21 – Sortie de `ip -br link show br0`

Le bridge br0 existe (MAC 9a:ee:05:93:7b:54). Son état est DOWN avec le drapeau NO-CARRIER alors qu'il a été activé (UP dans les drapeaux) : aucun port n'y est encore raccordé, donc il n'a pas de lien actif.

6.2 Créer deux « câbles » vers le switch

Listing 12 – Paires veth hôte ↔ switch

```
1 sudo ip link add h1-eth0 type veth peer name br-h1
2 sudo ip link add h2-eth0 type veth peer name br-h2
3 sudo ip link set h1-eth0 netns h1
4 sudo ip link set h2-eth0 netns h2
5 sudo ip link set br-h1 master br0
6 sudo ip link set br-h2 master br0
7 sudo ip link set br-h1 up
8 sudo ip link set br-h2 up
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link add h1-eth0 type veth peer name br-h1
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link add h2-eth0 type veth peer name br-h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 22 – Création des paires h1-eth0 / br-h1 et h2-eth0 / br-h2

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set h1-eth0 netns h1
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set h2-eth0 netns h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set br-h1 master br0
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set br-h2 master br0
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 23 – Déplacement de h1-eth0 et h2-eth0 dans leurs namespaces, puis rattachement de br-h1 et br-h2 au bridge

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set br-h1 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set br-h2 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ █
```

FIGURE 24 – Activation de br-h1 et br-h2

6.3 Adresser les hôtes

Listing 13 – Adressage des hôtes

```
1 sudo ip -n h1 addr add 10.20.0.1/24 dev h1-eth0
2 sudo ip -n h2 addr add 10.20.0.2/24 dev h2-eth0
3 sudo ip -n h1 link set lo up
4 sudo ip -n h2 link set lo up
5 sudo ip -n h1 link set h1-eth0 up
6 sudo ip -n h2 link set h2-eth0 up
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 addr add 10.20.0.1/24 dev h1-eth0
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 addr add 10.20.0.2/24 dev h2-eth0
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ █
```

FIGURE 25 – Attribution des adresses 10.20.0.1/24 et 10.20.0.2/24

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 link set lo up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 link set lo up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 link set h1-eth0 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 link set h2-eth0 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ █
```

FIGURE 26 – Activation de lo, h1-eth0 et h2-eth0

6.4 Vérifier le « câblage »

Listing 14 – Vérification du câblage

```
1 ip link show master br0
2 bridge link
3 sudo ip -n h1 -br addr
4 sudo ip -n h2 -br addr
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip link show master br0
9: br-h1@if10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br0 state UP mode DEFAULT group default qlen 1000
   link/ether 62:ba:e3:27:57:f4 brd ff:ff:ff:ff:ff:ff link-netns h1
11: br-h2@if12: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br0 state UP mode DEFAULT group default qlen 1000
   link/ether 4a:02:96:d0:8a:20 brd ff:ff:ff:ff:ff:ff link-netns h2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ bridge link
9: br-h1@if10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master br0 state forwarding priority 32 cost 2
11: br-h2@if12: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master br0 state forwarding priority 32 cost 2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h1 -br addr
[sudo] Mot de passe de hathouti :
lo                UNKNOWN        127.0.0.1/8 ::1/128
h1-eth0@if9       UP                10.20.0.1/24 fe80::6077:ccff:fe24:eb8c/64
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h2 -br addr
lo                UNKNOWN        127.0.0.1/8 ::1/128
h2-eth0@if11      UP                10.20.0.2/24 fe80::e88b:8dff:fe99:e7ef/64
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ █
```

FIGURE 27 – Vérification : ip link show master br0, bridge link et adresses de h1 et h2

br-h1 et br-h2 sont membres de br0 et en état forwarding (priority 32, cost 2). L'interface link-netns de chacun indique le namespace où se trouve l'autre bout du câble (h1 et h2). h1 et h2 sont UP avec 10.20.0.1/24 et 10.20.0.2/24.

6.5 Tester et observer la table MAC

Listing 15 – Test et table MAC du bridge

```
1 sudo ip netns exec h1 ping -c 3 10.20.0.2
2 bridge fdb show br br0
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h1 ping -c 3 10.20.0.2
PING 10.20.0.2 (10.20.0.2) 56(84) bytes of data.
64 bytes from 10.20.0.2: icmp_seq=1 ttl=64 time=0.123 ms
64 bytes from 10.20.0.2: icmp_seq=2 ttl=64 time=0.105 ms
64 bytes from 10.20.0.2: icmp_seq=3 ttl=64 time=0.082 ms

--- 10.20.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2069ms
rtt min/avg/max/mdev = 0.082/0.103/0.123/0.016 ms
```

FIGURE 28 – Ping de h1 vers h2 à travers le bridge

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ bridge fdb show br br0
33:33:00:00:00:01 dev br0 self permanent
01:00:5e:00:00:6a dev br0 self permanent
33:33:00:00:00:6a dev br0 self permanent
01:00:5e:00:00:01 dev br0 self permanent
33:33:ff:93:7b:54 dev br0 self permanent
33:33:00:00:00:fb dev br0 self permanent
9a:ee:05:93:7b:54 dev br0 vlan 1 master br0 permanent
9a:ee:05:93:7b:54 dev br0 master br0 permanent
62:77:cc:24:eb:8c dev br-h1 master br0
62:ba:e3:27:57:f4 dev br-h1 vlan 1 master br0 permanent
62:ba:e3:27:57:f4 dev br-h1 master br0 permanent
33:33:00:00:00:01 dev br-h1 self permanent
01:00:5e:00:00:01 dev br-h1 self permanent
33:33:ff:27:57:f4 dev br-h1 self permanent
33:33:00:00:00:fb dev br-h1 self permanent
ea:8b:8d:f9:e7:ef dev br-h2 master br0
4a:02:96:d0:8a:20 dev br-h2 vlan 1 master br0 permanent
4a:02:96:d0:8a:20 dev br-h2 master br0 permanent
33:33:00:00:00:01 dev br-h2 self permanent
01:00:5e:00:00:01 dev br-h2 self permanent
33:33:ff:d0:8a:20 dev br-h2 self permanent
33:33:00:00:00:fb dev br-h2 self permanent
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 29 – Table MAC du bridge : bridge fdb show br br0

Le ping réussit (3/3, RTT moyen 0,103 ms). La table `fdb` contient deux types d'entrées :

- des entrées **appries dynamiquement**, sans mention **permanent** : `62:77:cc:24:eb:8c` sur `br-h1` (MAC de `h1-eth0`) et `ea:8b:8d:f9:e7:ef` sur `br-h2` (MAC de `h2-eth0`);
- des entrées **permanent** : les MAC propres de `br0` et de ses ports, ainsi que des adresses multicast (`33:33:...`, `01:00:5e:...`).

C'est exactement le comportement d'un switch Ethernet classique : le bridge a appris, à partir des adresses source des trames vues, derrière quel port se trouve chaque hôte.

7 Analyse de trafic sur le bridge

Listing 16 – Capture sur `br0`

```
1 sudo tcpdump -ni br0 'arp or icmp'
2 sudo ip netns exec h1 ping -c 3 10.20.0.2
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h1 ping -c 3 10.20.0.2
PING 10.20.0.2 (10.20.0.2) 56(84) bytes of data.
64 bytes from 10.20.0.2: icmp_seq=1 ttl=64 time=0.087 ms
64 bytes from 10.20.0.2: icmp_seq=2 ttl=64 time=0.088 ms
64 bytes from 10.20.0.2: icmp_seq=3 ttl=64 time=0.101 ms

--- 10.20.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2055ms
rtt min/avg/max/mdev = 0.087/0.092/0.101/0.006 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 30 – Ping de `h1` vers `h2` (terminal B)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo tcpdump -ni br0 'arp or icmp'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on br0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
08:45:48.155220 IP 10.20.0.1 > 10.20.0.2: ICMP echo request, id 15849, seq 1, length 64
08:45:48.155256 IP 10.20.0.2 > 10.20.0.1: ICMP echo reply, id 15849, seq 1, length 64
08:45:49.186726 IP 10.20.0.1 > 10.20.0.2: ICMP echo request, id 15849, seq 2, length 64
08:45:49.186770 IP 10.20.0.2 > 10.20.0.1: ICMP echo reply, id 15849, seq 2, length 64
08:45:50.210694 IP 10.20.0.1 > 10.20.0.2: ICMP echo request, id 15849, seq 3, length 64
08:45:50.210731 IP 10.20.0.2 > 10.20.0.1: ICMP echo reply, id 15849, seq 3, length 64
08:45:53.410667 ARP, Request who-has 10.20.0.1 tell 10.20.0.2, length 28
08:45:53.410674 ARP, Request who-has 10.20.0.2 tell 10.20.0.1, length 28
08:45:53.410700 ARP, Reply 10.20.0.1 is-at 62:77:cc:24:eb:8c, length 28
08:45:53.410703 ARP, Reply 10.20.0.2 is-at ea:8b:8d:f9:e7:ef, length 28
^C
10 packets captured
10 packets received by filter
0 packets dropped by kernel
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 31 – Capture `tcpdump` sur `br0` (terminal A)

Observation. La capture montre les trois Echo Request / Echo Reply (seq 1 à 3) à partir de 08:45:48, puis à 08:45:53 deux `who-has` (un par hôte) avec leurs Reply :

10.20.0.1 is-at 62:77:cc:24:eb:8c et 10.20.0.2 is-at ea:8b:8d:f9:e7:ef. Ces MAC sont celles apprises par la fdb à la section précédente. Comme à la section 5, l'ARP n'est pas avant l'ICMP, car le ping de l'étape 6.5 avait déjà rempli les caches ; l'ARP vu ensuite est la revalidation DELAY/PROBE des entrées.

Questions de compréhension

1. Quel protocole apparaît avant ICMP lorsque le cache voisin est vide ?

ARP. L'hôte source diffuse un who-has pour connaître la MAC de la destination ; la réponse is-at lui permet ensuite d'envoyer les Echo Request. Ce comportement est observé au mini-défi 3, après `ip neigh flush`.

2. Le bridge travaille-t-il principalement avec des adresses IP ou des adresses MAC ?

Avec des adresses MAC. Le bridge est un équipement de niveau 2 : il décide du port de sortie d'après l'adresse MAC de destination de la trame, en consultant sa table fdb. Il ne connaît ni ne manipule les adresses IP ; `tcpdump` les affiche seulement parce qu'il décode le contenu des trames.

3. Quelle différence voyez-vous entre la table ARP/neighbor d'un hôte et la FDB du bridge ?

	Table neighbor (hôte)	FDB (bridge)
Associe	une adresse IP à une adresse MAC	une adresse MAC à un port
Niveau	L3 → L2	L2
Apprise par	échanges ARP (requêtes et réponses)	adresses MAC source des trames vues
Sert à	construire la trame Ethernet vers le prochain saut	choisir le port de sortie de la trame
Commande	<code>ip neigh</code>	<code>bridge fdb show</code>

TABLE 6 – Table neighbor et FDB

8 Faire circuler un vrai service HTTP

Listing 17 – Serveur Web dans h1, client dans h2

```

1 sudo ip netns exec h1 python3 -m http.server 8080 --bind
  10.20.0.1
2 sudo ip netns exec h2 curl http://10.20.0.1:8080

```

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h1 python3 -m http.server 8080 --bind 10.20.0.1
Serving HTTP on 10.20.0.1 port 8080 (http://10.20.0.1:8080/) ...
10.20.0.2 - - [30/Sep/2026 08:51:46] "GET / HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$

```

FIGURE 32 – Serveur HTTP lancé dans h1 (10.20.0.1:8080) et sa journalisation

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h2 curl http://10.20.0.1:8080
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href="Screens/">Screens/</a></li>
<li><a href="TP1_Reseau_Linux_Namespaces_Veth_Bridge_Preparation.pdf">TP1_Reseau_Linux_Namespaces_Veth_Bridge_Preparation.pdf</a></li>
</ul>
<hr>
</body>
</html>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 33 – Requête curl depuis h2

h2 reçoit une page HTML (liste du répertoire). Côté serveur, le journal confirme la requête : 10.20.0.2 -- -- "GET / HTTP/1.1" 200, l'adresse source étant celle de h2. Une application cliente et un serveur, situés dans deux hôtes réseau virtuels différents, ont donc communiqué à travers le bridge.

Le listing reçu affiche les fichiers du dossier depuis lequel le serveur a été lancé (Screens/ et le PDF du sujet) : le namespace n'isole pas le système de fichiers, seulement la pile réseau.

Pourquoi c'est important pour Mininet. Dans Mininet, on lancera de la même façon ping, iperf, Python, curl ou d'autres programmes « dans » h1, h2, etc. : les hôtes Mininet exécutent de vrais processus Linux.

9 Mini-défis

9.1 Défi 1 — ajouter un troisième namespace h3 connecté à br0

Listing 18 – Création de h3

```
1 sudo ip netns add h3
2 sudo ip link add h3-eth0 type veth peer name br-h3
3 sudo ip link set h3-eth0 netns h3
4 sudo ip link set br-h3 master br0
5 sudo ip link set br-h3 up
6 sudo ip -n h3 addr add 10.20.0.3/24 dev h3-eth0
7 sudo ip -n h3 link set lo up
8 sudo ip -n h3 link set h3-eth0 up
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns add h3
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link add h3-eth0 type veth peer name br-h3
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 34 – Création du namespace h3 et de la paire h3-eth0 / br-h3

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set h3-eth0 netns h3
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set br-h3 master br0
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link set br-h3 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 35 – Déplacement de h3-eth0 dans h3, rattachement de br-h3 à br0 et activation

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h3 addr add 10.20.0.3/24 dev h3-eth0
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h3 link set lo up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h3 link set h3-eth0 up
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 36 – Adressage de h3 (10.20.0.3/24) et activation de ses interfaces

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ bridge link
9: br-h1@if10: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master br0 state forwarding priority 32 cost 2
11: br-h2@if12: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master br0 state forwarding priority 32 cost 2
13: br-h3@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master br0 state forwarding priority 32 cost 2
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h3 -br addr
lo                UNKNOWN      127.0.0.1/8 ::1/128
h3-eth0@if13      UP          10.20.0.3/24 fe80::cd3:caff:fe97:9121/64
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 37 – Vérification avec bridge link et ip -n h3 -br addr

bridge link liste maintenant trois ports en forwarding (br-h1, br-h2, br-h3) et h3-eth0 est UP avec 10.20.0.3/24.

9.2 Défi 2 — vérifier que h3 joint h1 et h2

Listing 19 – Tests depuis h3

```
1 sudo ip netns exec h3 ping -c 2 10.20.0.1
2 sudo ip netns exec h3 ping -c 2 10.20.0.2
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h3 ping -c 2 10.20.0.1
PING 10.20.0.1 (10.20.0.1) 56(84) bytes of data.
64 bytes from 10.20.0.1: icmp_seq=1 ttl=64 time=0.110 ms
64 bytes from 10.20.0.1: icmp_seq=2 ttl=64 time=0.136 ms

--- 10.20.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1020ms
rtt min/avg/max/mdev = 0.110/0.123/0.136/0.013 ms
```

FIGURE 38 – Ping de h3 vers h1 (10.20.0.1)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h3 ping -c 2 10.20.0.2
PING 10.20.0.2 (10.20.0.2) 56(84) bytes of data.
64 bytes from 10.20.0.2: icmp_seq=1 ttl=64 time=0.172 ms
64 bytes from 10.20.0.2: icmp_seq=2 ttl=64 time=0.078 ms

--- 10.20.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1025ms
rtt min/avg/max/mdev = 0.078/0.125/0.172/0.047 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 39 – Ping de h3 vers h2 (10.20.0.2)

h3 joint h1 et h2 (2/2 réponses, 0% de perte dans les deux cas) : les trois hôtes sont dans le même sous-réseau 10.20.0.0/24 et sur le même bridge, sans routeur.

9.3 Défi 3 — vider le cache neighbor de h3 et observer le ping suivant

Listing 20 – Capture sur br0 après neigh flush

```
1 sudo tcpdump -ni br0 'arp or icmp'
2 sudo ip -n h3 neigh flush all
3 sudo ip netns exec h3 ping -c 2 10.20.0.1
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h3 neigh flush all
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h3 ping -c 2 10.20.0.1
PING 10.20.0.1 (10.20.0.1) 56(84) bytes of data.
64 bytes from 10.20.0.1: icmp_seq=1 ttl=64 time=0.132 ms
64 bytes from 10.20.0.1: icmp_seq=2 ttl=64 time=0.099 ms

--- 10.20.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1034ms
rtt min/avg/max/mdev = 0.099/0.115/0.132/0.016 ms
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 40 – `ip -n h3 neigh flush all` puis ping de h3 vers h1 (terminal B)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo tcpdump -ni br0 'arp or icmp'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on br0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:02:27.192676 ARP, Request who-has 10.20.0.1 tell 10.20.0.3, length 28
09:02:27.192700 ARP, Reply 10.20.0.1 is-at 62:77:cc:24:eb:8c, length 28
09:02:27.192715 IP 10.20.0.3 > 10.20.0.1: ICMP echo request, id 18140, seq 1, length 64
09:02:27.192734 IP 10.20.0.1 > 10.20.0.3: ICMP echo reply, id 18140, seq 1, length 64
09:02:28.226672 IP 10.20.0.3 > 10.20.0.1: ICMP echo request, id 18140, seq 2, length 64
09:02:28.226716 IP 10.20.0.1 > 10.20.0.3: ICMP echo reply, id 18140, seq 2, length 64
09:02:32.322514 ARP, Request who-has 10.20.0.3 tell 10.20.0.1, length 28
09:02:32.322535 ARP, Reply 10.20.0.3 is-at 0e:d3:ca:97:91:21, length 28
^C
8 packets captured
8 packets received by filter
0 packets dropped by kernel
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 41 – Capture tcpdump sur br0 (terminal A)

Observation. Cette fois l'enchaînement attendu apparaît :

1. `who-has 10.20.0.1 tell 10.20.0.3`, requête ARP émise par h3 (cache vide) ;
2. `Reply 10.20.0.1 is-at 62:77:cc:24:eb:8c`, réponse de h1 ;
3. les Echo Request / Echo Reply ICMP (seq 1 et seq 2) partent alors vers la bonne destination.

Environ 5 secondes plus tard, un dernier échange ARP apparaît dans l'autre sens (`who-has 10.20.0.3 tell 10.20.0.1`, réponse `10.20.0.3 is-at 0e:d3:ca:97:91:21`). C'est h1 qui revalide son entrée pour h3 : elle avait été créée à la réception de la requête ARP de h3 (donc non confirmée) et utilisée pour envoyer l'Echo Reply, ce qui la fait passer par DELAY puis PROBE. h3, lui, n'a pas de nouvel ARP à faire, car sa propre entrée a été confirmée par la réponse ARP qu'il vient de recevoir.

9.4 Défi 4 — h3 avec l'adresse 10.21.0.3/24, sans routeur

Listing 21 – Changement d'adresse de h3

```
1 sudo ip -n h3 addr flush dev h3-eth0
2 sudo ip -n h3 addr add 10.21.0.3/24 dev h3-eth0
3 sudo ip netns exec h3 ping -c 2 10.20.0.1
4 sudo ip -n h3 route
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h3 addr flush dev h3-eth0
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h3 addr add 10.21.0.3/24 dev h3-eth0
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 42 – Remplacement de l'adresse de h3 par 10.21.0.3/24

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns exec h3 ping -c 2 10.20.0.1
ping: connect: Le réseau n'est pas accessible
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 43 – Ping de h3 vers h1 : réseau inaccessible

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip -n h3 route
10.21.0.0/24 dev h3-eth0 proto kernel scope link src 10.21.0.3
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$
```

FIGURE 44 – Table de routage de h3 : `ip -n h3 route`

Résultat. Le ping échoue immédiatement avec le message « Le réseau n'est pas accessible » (*Network is unreachable*). La table de routage de h3 ne contient qu'une seule route : `10.21.0.0/24 dev h3-eth0 proto kernel scope link src 10.21.0.3`.

Explication.

- h3 est toujours branché sur le même bridge, donc dans le même domaine de diffusion Ethernet que h1 et h2. Mais 10.21.0.3/24 appartient à un sous-réseau IP **différent** de 10.20.0.0/24 ;
- pour h3, 10.20.0.1 n'est pas dans son réseau local et il n'a pas de route par défaut : le noyau refuse d'envoyer le paquet, sans même émettre d'ARP ;
- dans l'autre sens, h1 et h2 n'ont aucune route vers 10.21.0.0/24 non plus ;
- le bridge n'y change rien : il commute des trames Ethernet mais ne connaît pas les réseaux IP. Pour faire communiquer deux sous-réseaux, il faut un **routeur** (niveau 3), par exemple un namespace avec `ip_forward=1` et une interface dans chaque sous-réseau.

10 Nettoyage obligatoire

Listing 22 – Nettoyage de l’environnement

```

1 sudo ip netns del h1 2>/dev/null || true
2 sudo ip netns del h2 2>/dev/null || true
3 sudo ip netns del h3 2>/dev/null || true
4 sudo ip link del br0 2>/dev/null || true
5 ip netns list
6 ip -br link | grep -E 'br0|br-h|h[123]-eth' || true

```

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns del h1 2>/dev/null || true
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns del h2 2>/dev/null || true
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip netns del h3 2>/dev/null || true
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ sudo ip link del br0 2>/dev/null || true
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip netns list
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$ ip -br link | grep -E 'br0|br-h|h[123]-eth' || true
virbr0                DOWN                52:54:00:9e:3b:49 <NO-CARRIER,BROADCAST,MULTICAST,UP>
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP1$

```

FIGURE 45 – Nettoyage final et vérification

`ip netns list` ne renvoie rien : plus aucun namespace `h1`, `h2` ou `h3`. Le `grep` ne remonte qu’une ligne, `virbr0`, qui n’est qu’une fausse correspondance avec le motif `br0` : c’est le pont libvirt déjà présent avant le TP (voir section 3), pas le bridge `br0` du TP. Aucun élément du TP ne subsiste.

11 Synthèse : le lien direct avec Mininet

Ce que j’ai créé	Rôle	Ce que Mininet fera ensuite
network namespace	Un hôte réseau isolé	<code>h1</code> , <code>h2</code> , <code>h3</code> ...
paire veth	Un lien/câble virtuel	liens hôte ↔ switch
bridge <code>br0</code>	Commutation Ethernet L2	switch virtuel, souvent Open vSwitch (OVS)
<code>ip</code> / <code>ping</code> / <code>tcpdump</code>	Configuration et diagnostic	mêmes outils depuis la CLI Mininet

TABLE 7 – Correspondance entre le TP et Mininet

Réponse à la question fil rouge. Pour faire communiquer deux « machines » dans un seul Ubuntu sans créer de VM, on crée un **namespace** par machine (pile réseau isolée), on relie chacun à un **switch logiciel** (`br0`) par une **paire veth** (un câble virtuel), puis on leur donne des adresses du même sous-réseau. Les hôtes exécutent alors de vrais processus Linux (`ping`, serveur HTTP, `curl`) qui communiquent à travers le bridge.

Conclusion. Mininet ne crée pas un nouveau monde : il automatise massivement des mécanismes Linux que je viens de construire à la main.

12 Grille de validation rapide

Vérification	Commande / observation	OK
Namespaces créés et inspectés	<code>ip netns list; ip -n h1 -br addr</code>	☒
Paire veth comprise	ping direct <code>h1 ↔ h2</code> (étape A)	☒
Bridge fonctionnel	<code>ip link show master br0; bridge link</code>	☒
ARP/ICMP observés	<code>tcpdump -ni br0</code> (sections 5, 7 et défi 3)	☒
Service applicatif testé	<code>python3 -m http.server</code> + <code>curl</code>	☒
Environnement nettoyé	aucun <code>h1/h2/br0</code> restant	☒

TABLE 8 – Grille de validation